SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1418 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: **Total Marks:50**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I K NISHALINI (192521301) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K NISHALINI

Annexure A

ANALYTICAL PROBLEM SOLVING

1. Trace the compilation process
Consider the following C statement:
total = price + quantity * 10;
Trace this statement through the relevant phases of a compiler by identifying the lexemes and tokens produced during lexical analysis, explaining how the syntax analyzer uses the token stream, describing the role of semantic analysis, and showing how an appropriate intermediate representation could be generated. Finally, explain how the grouping of compiler phases into front end, middle end, and back end helps in processing this statement.

1. Problem Understanding

We must follow one C assignment statement through the phases of a compiler: lexical analysis, syntax analysis, semantic analysis, intermediate code generation, and finally explain the front end / middle end / back end grouping. Assumption: total, price and quantity have been declared earlier as int.

2. Method

Phase-by-phase tracing of the compiler: Source code → Lexical Analyzer → Syntax Analyzer → Semantic Analyzer → Intermediate Code Generator → (Optimizer) → Code Generator.

3. Solution Process

Phase 1 – Lexical Analysis (Scanner)

The scanner reads the character stream left to right, ignores whitespace, and groups characters into lexemes. Each lexeme is mapped to a token (token-name, attribute-value).

No.	Lexeme	Token	Attribute / Remark
1	total	IDENTIFIER	pointer to symbol-table entry of total
2	=	ASSIGN_OP	assignment operator
3	price	IDENTIFIER	symbol-table entry of price
4	+	ARITH_OP (ADD)	addition
5	quantity	IDENTIFIER	symbol-table entry of quantity
6	*	ARITH_OP (MUL)	multiplication
7	10	INTEGER_CONSTANT	value = 10
8	;	SEMICOLON	statement terminator

Token stream: <id,1> <=> <id,2> <+> <id,3> <*> <num,10> <;> (8 tokens)

Phase 2 – Syntax Analysis (Parser)

The parser takes the token stream (not the characters) and checks it against the grammar of the language. It builds a parse tree / syntax tree that also fixes the operator precedence and associativity: * has higher precedence than +, so quantity * 10 is evaluated first. A suitable grammar is:

```
stmt   -> id = expr ;
expr   -> expr + term | term
term   -> term * factor | factor
factor -> id | num | ( expr )
```

Derivation (leftmost): stmt ⇒ id = expr ; ⇒ id = expr + term ; ⇒ id = term + term ; ⇒ id = factor + term ; ⇒ id = id + term ; ⇒ id = id + term * factor ; ⇒ id = id + factor * factor ; ⇒ id = id + id * num ;

Abstract syntax tree produced:

```
        =
       / \
   total   +
         / \
    price     *
            / \
     quantity   10
```

If any token is out of place (for example, total = price + * 10;), the parser reports a syntax error.

Phase 3 – Semantic Analysis

The semantic analyzer walks the syntax tree and checks the meaning of the statement using the symbol table:

- Declaration check: total, price and quantity must be declared before use.
- Type checking: price (int) + [quantity (int) * 10 (int)] → int; the result is assigned to total (int) – types are compatible.
- Type coercion: if price were float, the analyzer inserts an implicit conversion, e.g. inttofloat(quantity * 10), and the tree is annotated accordingly.
- L-value check: the left side of = must be a modifiable variable (total is).

Phase 4 – Intermediate Code Generation (Three-Address Code)

```
t1 = quantity * 10
t2 = price + t1
total = t2
```

Quadruple representation:

#	**Operator**	**Arg1**	**Arg2**	**Result**
0	*	quantity	10	t1
1	+	price	t1	t2
2	=	t2	–	total

(Optimization can fold the last two lines into t1 = quantity * 10; total = price + t1.)

Front End, Middle End and Back End

Part	**Phases**	**Role for total = price + quantity * 10;**
Front End (source-language dependent, machine independent)	Lexical, Syntax, Semantic analysis, Intermediate code generation	Tokenizes the 8 lexemes, builds the tree with correct precedence, checks types, produces t1, t2 three-address code.
Middle End (machine independent)	Code optimization on the IR	Removes the redundant temporary (t2), applies constant folding / strength reduction, e.g. multiplication by 10 can become shifts and adds.
Back End (target-machine dependent)	Code generation, register allocation, machine-dependent optimization	Maps IR to target instructions, e.g. LOAD R1, quantity; MUL R1, #10; ADD R1, price; STORE total, R1.

Benefit of grouping: the same IR links m languages to n machines with only m + n components instead of m × n compilers; the optimizer is written once, and a new language only needs a new front end or a new CPU only a new back end.

4. Accuracy

8 lexemes → 8 tokens (3 identifiers, 1 constant, 2 operators, 1 assignment, 1 terminator); 2 operations in the IR (one * and one +) plus one copy; evaluation order = quantity * 10 first, then + price.

5. Interpretation

Each phase transforms the same statement into a more refined form – characters → tokens → tree → type-checked tree → three-address code → machine code. Errors are caught as early as possible (lexical/syntax/semantic), and the front/middle/back-end split makes the compiler portable and reusable.

2. Analyze token recognition and ambiguity
Consider the following input statement:
int count = a+++b;
Analyze how a lexical analyzer should divide the input into lexemes and tokens. Identify the possible ambiguity in recognizing the sequence +++, explain how longest-match/token recognition rules influence the result, and propose suitable token specifications for identifiers, keywords, assignment operators, increment operators, and arithmetic operators. Justify why your proposed tokenization is appropriate for the given language.

1. Problem Understanding

The challenge is that the sequence +++ has no spaces and can be split in more than one way: (++) (+) or (+) (++). The lexer must apply one deterministic rule to decide.

2. Method

Longest-match (maximal munch) rule: at each position the lexer takes the longest prefix of the remaining input that matches any token pattern. If two patterns match the same length, the pattern listed first wins (this is how keywords beat identifiers).

3. Solution Process

Scanning left to right:

Position	Remaining input	Longest match	Token
1	int count = a+++b;	int (keyword rule precedes identifier rule)	KEYWORD
2	count = a+++b;	count	IDENTIFIER
3	= a+++b;	= (next char is a space, so == is not matched)	ASSIGN_OP
4	a+++b;	a	IDENTIFIER
5	+++b;	++ (2 chars) beats + (1 char); +++ is not a token in C	INCREMENT_OP
6	+b;	+	ARITH_OP (ADD)
7	b;	b	IDENTIFIER
8	;	;	SEMICOLON

Final lexemes/tokens (8): int | count | = | a | ++ | + | b | ;

The ambiguity

- Interpretation A: a ++ + b → (a++) + b (chosen by the lexer)
- Interpretation B: a + ++ b → a + (++b) (never produced, because the lexer has already consumed ++ at the first + position)

The lexer does not look ahead beyond the longest match nor does it backtrack to find a 'better' parse, so the result is deterministic: count = (a++) + b. Meaning: use the old value of a in the sum, then increment a. To obtain interpretation B the programmer must write a + ++b.

Token specifications (regular expressions)

Category	Regular expression	Examples
Keyword	int | float | char | if | else | while | return	int
Identifier	[A-Za-z_][A-Za-z0-9_]*	count, a, b
Assignment operator	= | += | -= | *= | /=	=
Increment / decrement	++ | --	++
Arithmetic operator	+ | - | * | / | %	+
Separator	; | , | (|) | { | }	;

Priority: keywords before identifiers; ++ / -- / += before the single-character + / - (longer patterns win automatically by maximal munch).

4. Accuracy

Input has 8 tokens. Characters consumed: 3 + 5 + 1 + 1 + 2 + 1 + 1 + 1 = 15 non-space characters (int, count, =, a, ++, +, b, ;) – all accounted for.

5. Interpretation / Justification

Maximal munch is appropriate because it is simple, fast (single left-to-right pass with no backtracking), and matches the C language definition, so the compiler behaves exactly as the language standard requires. It also keeps operators such as ++, <=, == from being broken into smaller tokens, and the syntax analyzer then receives an unambiguous token stream. Writing a + ++b or a++ + b with spaces removes any doubt for human readers.

3. Determine and classify tokens
Consider the following program:

```
/* Calculate the average */
int main()
{
   int sum = 100, count = 5;
   float average = sum / count;
   return 0;
}
```

Determine the total number of tokens generated by the lexical analyzer. Analyze the program line by line and classify each token as a keyword, identifier, constant/literal, operator, separator, or punctuation symbol. Explain how comments and whitespace are handled by the lexical analyzer and justify the final token count.

1. Problem Understanding

Count all tokens the scanner passes to the parser and classify each one. Comments and whitespace are not tokens.

2. Method

Scan line by line, apply the longest-match rule, discard the comment and whitespace, and classify each lexeme.

3. Solution Process – Line-by-Line Analysis

Line	Source	Tokens (classification)	Count
1	/* Calculate the average */	Comment – discarded	0
2	int main()	int (Keyword), main (Identifier), ((Separator),) (Separator)	4
3	{	{ (Separator)	1

Line	Source	Tokens (classification)	Count
4	int sum = 100, count = 5;	int (Keyword), sum (Identifier), = (Operator), 100 (Constant), , (Punctuation), count (Identifier), = (Operator), 5 (Constant), ; (Punctuation)	9
5	float average = sum / count;	float (Keyword), average (Identifier), = (Operator), sum (Identifier), / (Operator), count (Identifier), ; (Punctuation)	7
6	return 0;	return (Keyword), 0 (Constant), ; (Punctuation)	3
7	}	} (Separator)	1
		TOTAL	25

Classification Summary

Category	Tokens	Count
Keywords	int, int, float, return	4
Identifiers	main, sum, count, average, sum, count	6
Constants / Literals	100, 5, 0	3
Operators	=, =, =, /	4
Separators	( ) { }	4
Punctuation	, ; ; ;	4
TOTAL		25

Handling of Comments and Whitespace

- The comment /* ... */ is recognised by the pattern /*([^*]|*+[^*/])**+/ ; its action is empty, so no token is returned.
- Whitespace (blank, tab, newline) matches [\t\n]+ and is skipped; it only separates lexemes (e.g. int and main) and is never passed to the parser.
- Because they are not part of the grammar, they do not increase the token count. The scanner may still count newlines to report line numbers in error messages.

4. Accuracy

4 + 6 + 3 + 4 + 4 + 4 = 25 and by lines 4 + 1 + 9 + 7 + 3 + 1 = 25. Both methods agree, so the total number of tokens = 25.

5. Interpretation

Only meaningful lexical units reach the parser; 25 tokens represent the whole function. Note that 'int' appears twice and identifiers sum/count appear twice – each occurrence is a separate token, though they share one symbol-table entry per identifier.

4. Design token specifications using regular expressions

A simplified programming language contains the following token categories:

- **Keywords: if, else, while, return**
- **Identifiers: begin with a letter and may be followed by letters or digits**
- **Integer constants: one or more digits**
- **Relational operators: <, <=, >, >=, ==, !=**
- **Arithmetic operators: +, -, *, /**
- **Separators: (,), {, }, ;**

For the input:

```
if (count >= 10) {
   total = count + 25;
   return total;
}
```

design appropriate regular-expression specifications for the token categories, trace the input through the lexical analyzer, and produce the complete token stream. Explain how the lexical analyzer distinguishes keywords from identifiers and how overlapping operators such as < and <= are recognized correctly.

1. Problem Understanding

Define regular expressions for each token class, then scan the program and list the tokens. Observation: the input contains the assignment symbol =, which is not in the given operator list, so a rule ASSIGN → = is added to the specification.

2. Method

Regular definitions: letter = [A-Za-z], digit = [0-9]. Patterns are tried with longest-match; on a tie the earlier rule (keywords) wins.

3. Solution Process

Regular-Expression Specification

Token	**Regular expression**
KEYWORD	if | else | while | return
IDENTIFIER	letter (letter | digit)*
INTEGER CONSTANT	digit+ (i.e. digit digit*)
RELATIONAL OPERATOR	< | <= | > | >= | == | !=
ARITHMETIC OPERATOR	+ | - | * | /
ASSIGNMENT (added)	=
SEPARATOR	(|) | { | } | ;
WHITESPACE (skipped)	(blank | tab | newline)+

Trace of the Input

#	Lexeme	Matched rule	Token
1	if	Keyword (matches both KEYWORD and IDENTIFIER, keyword listed first)	KEYWORD
2	(	Separator	SEPARATOR
3	count	letter(letter|digit)*	IDENTIFIER
4	>=	longest match: >= (2) over > (1)	RELOP
5	10	digit+	INTEGER
6	)	Separator	SEPARATOR
7	{	Separator	SEPARATOR
8	total	Identifier	IDENTIFIER
9	=	Assignment (next char is a space, so not ==)	ASSIGN
10	count	Identifier	IDENTIFIER
11	+	Arithmetic operator	ARITH_OP
12	25	digit+	INTEGER
13	;	Separator	SEPARATOR
14	return	Keyword	KEYWORD
15	total	Identifier	IDENTIFIER
16	;	Separator	SEPARATOR
17	}	Separator	SEPARATOR

Complete Token Stream

> <KEYWORD,if> <SEP,(> <ID,count> <RELOP,>=> <INT,10> <SEP,)> <SEP,{>
> <ID,total> <ASSIGN,=> <ID,count> <ARITH,+> <INT,25> <SEP,;>
> <KEYWORD,return> <ID,total> <SEP,;>
> <SEP,}>

How keywords are distinguished from identifiers

The pattern for identifiers also matches the words if, else, while and return. The scanner resolves this in one of two ways: (a) in LEX/regex style, the keyword rules are written before the identifier rule, so for an equal-length match the first rule wins; (b) the scanner recognises the word as an identifier and looks it up in a reserved-word (keyword) table; if found, a keyword token is returned. A longer word such as ifx or returned is longer than the keyword, so longest match makes it an identifier.

How overlapping operators such as < and <= are recognised

After reading '<' the DFA enters a state that is accepting for <. It peeks at the next character: if it is '=', it moves to a second accepting state and returns <=. Otherwise it retracts (lookahead is not consumed) and returns <. The same logic applies to >, >=, = and ==, and for != the character '!' alone is not a valid token. This is maximal munch in action: >= (2 characters) is preferred over > followed by =.

```
          '='
(start) --'<'--> (S1: accept '<') --'='--> (S2: accept '<=')
                      |
              other char: retract, return '<'
```

4. Accuracy

Total tokens = 17 (keywords 2, identifiers 4, integer constants 2, relational 1, arithmetic 1, assignment 1, separators 6).

5. Interpretation

The regular expressions are precise and non-overlapping once priority and longest-match rules are applied, so the input is tokenized uniquely. The resulting token stream can be handed directly to the parser.

5. Analyze and implement a LEX specification
A college wants to develop a lexical analyzer using LEX for a simple language containing:

- **Keywords: int, float, if, else, return**
- **Identifiers**
- **Integer constants**
- **Floating-point constants**
- **Arithmetic operators: +, -, *, /**
- **Relational operators: <, <=, >, >=, ==**
- **Assignment operator: =**
- **Separators: (,), {, }, ;**

For the input:
float average = total / 5.0;
if (average >= 50) return average;
Analyze the tokenization requirements and develop a LEX specification containing suitable regular-expression patterns and actions for recognizing the above token categories. Explain the purpose of the declaration section, rule section, and auxiliary procedure section of the LEX program. Finally, describe the sequence of commands required to generate lex.yy.c, compile it, and execute the resulting lexical analyzer.

1. Problem Understanding / Tokenization Requirements

Token class	Pattern needed	Notes
Keywords	int, float, if, else, return	Must appear before identifier rule
Identifiers	[a-zA-Z_][a-zA-Z0-9_]*	letters, digits, underscore

Token class	Pattern needed	Notes
Integer constants	[0-9]+	e.g. 50
Floating constants	[0-9]+\.[0-9]+	e.g. 5.0 – must be recognised before integer, or longest match handles it
Arithmetic operators	[+\-*/]	
Relational operators	<=, >=, ==, <, >	two-character first
Assignment	=	single character
Separators	[(){};]	
Whitespace	[\t\n]+	ignored

Expected token stream (16 tokens): float, average, =, total, /, 5.0, ; | if, (, average, >=, 50,), return, average, ;

2. Method & Solution – LEX Program (lexer.l)

```
%{
#include <stdio.h>
%}

digit    [0-9]
letter   [a-zA-Z_]
id       {letter}({letter}|{digit})*
int_no   {digit}+
float_no {digit}+\.{digit}+

%%
"int"|"float"|"if"|"else"|"return"   { printf("KEYWORD\t\t%s\n", yytext); }
{id}                { printf("IDENTIFIER\t%s\n", yytext); }
{float_no}          { printf("FLOAT_CONSTANT\t%s\n", yytext); }
{int_no}            { printf("INT_CONSTANT\t%s\n", yytext); }
"<="|">="|"=="|"<"|">"   { printf("RELATIONAL_OP\t%s\n", yytext); }
"="                 { printf("ASSIGN_OP\t%s\n", yytext); }
[+\-*/]             { printf("ARITH_OP\t%s\n", yytext); }
[(){};]             { printf("SEPARATOR\t%s\n", yytext); }
[ \t\n]+            { /* skip whitespace */ }
.                   { printf("UNKNOWN\t\t%s\n", yytext); }
%%

int yywrap(void) { return 1; }

int main(void)
{
   yylex();
   return 0;
}
```

Output for the given input

```
KEYWORD        float
IDENTIFIER     average
ASSIGN_OP      =
IDENTIFIER     total
ARITH_OP       /
FLOAT_CONSTANT  5.0
SEPARATOR      ;
KEYWORD        if
SEPARATOR      (
IDENTIFIER     average
RELATIONAL_OP  >=
INT_CONSTANT   50
SEPARATOR      )
KEYWORD        return
IDENTIFIER     average
SEPARATOR      ;
```

Why the rules are ordered this way

- Keywords before {id}: for 'float' both rules match 5 characters; LEX picks the rule listed first.
- 5.0 as one token: {float_no} matches 3 characters ('5.0') while {int_no} matches only 1 ('5'); LEX's longest-match rule selects the float rule.
- >= as one token: the 2-character pattern is longer than '>' alone, so the correct operator is returned.
- Catch-all '.' rule last: reports illegal characters instead of silently echoing them.

Structure of a LEX Program

Section	**Purpose**	**In this program**
Declaration (definition) section %{ ... %} and definitions	C headers, global variables, and named regular definitions (name → pattern) that make rules readable. Code inside %{ %} is copied unchanged to lex.yy.c.	#include <stdio.h>, digit, letter, id, int_no, float_no
Rules (translation rules) section between %% and %%	Each line is pattern { action }; when the scanner matches the pattern, the C action runs (print token, return token code, etc.).	Keyword, identifier, constant, operator, separator rules and the whitespace-skip rule
Auxiliary (user subroutines) section after the second %%	Ordinary C code copied to lex.yy.c: main(), yywrap(), helper functions (symbol table, error routines).	yywrap() returns 1 at end of input; main() calls yylex()

Commands to Generate, Compile and Run

Step	Command	Result
1. Write the specification	gedit lexer.l (or any editor)	LEX source file lexer.l
2. Generate the C scanner	lex lexer.l (or flex lexer.l)	lex.yy.c containing yylex()
3. Compile	gcc lex.yy.c -o lexer -lfl (use -ll with classic lex; not needed when yywrap and main are defined)	executable file lexer
4. Run	./lexer < input.txt (or ./lexer and type the input, finish with Ctrl+D)	token list printed on screen

Workflow: lexer.l → [lex/flex] → lex.yy.c → [gcc] → lexer (a.out) → input text → tokens.

4. Accuracy

Input produces 16 tokens: 3 keywords (float, if, return) + 4 identifiers (average, total, average, average) + 2 constants (5.0 float, 50 int) + 3 operators (=, /, >=) + 4 separators (;, (,), ;) = 16.

5. Interpretation

The specification recognises every required category without ambiguity: keyword priority, longest match for 5.0 and >=, and whitespace discarded. The LEX tool automatically converts these regular expressions into a DFA-driven C function (yylex), so adding new tokens requires only new rules.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation